# Models with Economic Indicators

Dr. Skip Crooker MBA 775 University of Nevada, Las Vegas

Thursday, October 22, 2026

## Table of contents

Naming the series on FRED . . . . . 1
Downloading from FRED . . . . . 3
Reconciling the frequencies . . . . . 4
Looking at the indicators . . . . . 6
What to take from this . . . . . 9

**AI Prompt**

Can you provide a list of commonly used economic indicators by economic forecasters? Please display the list in a table and group the indicators by their indicator grouping (e.g., Financial, Real Economy, Labor Market, Sentiment and Composite).

| Category | Leading Indicators |
|---|---|
| **Financial Market Indicators** | • Stock Market Indexes (e.g., S&P 500)<br>- Interest Rate Spreads (Yield Curve Slope)<br>- Money Supply (M2, inflation-adjusted) |
| **Real Economy Indicators** | • Housing Permits and Starts<br>- New Orders for Durable Goods<br>- Manufacturers' New Orders for Consumer Goods & Materials<br>- Average Weekly Hours in Manufacturing |
| **Labor Market Indicators** | • Initial Claims for Unemployment Insurance<br>- Temporary Help Employment |
| **Sentiment Indicators** | • Consumer Confidence & Expectations<br>- Purchasing Managers' Index (PMI) / ISM New Orders |
| **Composite Measures** | • Conference Board Leading Economic Index (LEI) |

## Naming the series on FRED

**AI Prompt**

Can you help me identify these leading market indicator time series variable names as used on the St. Louis Federal Reserve Economic Database website? Please prepare them as a **Python** dictionary I can use in my script. I will download these variables from FRED's public CSV endpoint, and I also want each entry to record the series' publication frequency, because I will have to reconcile daily, weekly, and monthly data before I can model with them.

The R version of this note stored the series IDs in a named character vector and downloaded them with `quantmod::getSymbols()`. In Python we use a dictionary of dictionaries so that each series carries its own metadata. The frequency field is not decoration — it is the thing that will bite us in a few cells.

```
fred_series = {
    # --- Financial Market ---------------------------------------------
    "SP500":     {"group": "Financial",  "freq": "D", "label": "S&P 500 index"},
    "T10Y3M":    {"group": "Financial",  "freq": "D", "label": "10Y - 3M Treasury spread"},
    "M2REAL":    {"group": "Financial",  "freq": "M", "label": "Real M2 money stock"},

    # --- Housing / Real Economy ---------------------------------------
    "PERMIT":    {"group": "Real",       "freq": "M", "label": "Building permits (SAAR)"},
    "HOUST":     {"group": "Real",       "freq": "M", "label": "Housing starts (SAAR)"},
    "DGORDER":   {"group": "Real",       "freq": "M", "label": "New orders: durable goods"},
    "UCOGNO":    {"group": "Real",       "freq": "M", "label": "New orders: consumer goods"},
    "AWHMAN":    {"group": "Real",       "freq": "M", "label": "Avg weekly hours, mfg"},

    # --- Labor Market -------------------------------------------------
    "ICSA":      {"group": "Labor",      "freq": "W", "label": "Initial jobless claims (SA)"},
    "TEMPHELPS": {"group": "Labor",      "freq": "M", "label": "Temporary help services (SA)"},

    # --- Sentiment ----------------------------------------------------
    "UMCSENT":   {"group": "Sentiment",  "freq": "M", "label": "U. Michigan consumer sentiment"},

    # --- Composite ----------------------------------------------------
    "USSLIND":   {"group": "Composite",  "freq": "M", "label": "Leading Index for the U.S."},
}

catalog = pd.DataFrame(fred_series).T
catalog.index.name = "series_id"
catalog
```

| series_id | group | freq | label |
|---|---|---|---|
| SP500 | Financial | D | S&P 500 index |
| T10Y3M | Financial | D | 10Y - 3M Treasury spread |
| M2REAL | Financial | M | Real M2 money stock |
| PERMIT | Real | M | Building permits (SAAR) |
| HOUST | Real | M | Housing starts (SAAR) |
| DGORDER | Real | M | New orders: durable goods |
| UCOGNO | Real | M | New orders: consumer goods |
| AWHMAN | Real | M | Avg weekly hours, mfg |
| ICSA | Labor | W | Initial jobless claims (SA) |
| TEMPHELPS | Labor | M | Temporary help services (SA) |
| UMCSENT | Sentiment | M | U. Michigan consumer sentiment |
| USSLIND | Composite | M | Leading Index for the U.S. |

## Downloading from FRED

FRED publishes every series at a public CSV endpoint that needs no API key, and the course module `fred_tools.py` wraps it. We use `fred_frame`, which fetches each series **independently** and reports what happened. That matters: a single bulk request for a dozen identifiers fails entirely if any one of them is misspelled or retired, which is a poor way to learn that one identifier was wrong.

```
START = "1990-01-01"

raw, status = fred_frame(fred_series.keys(), start=START)

if raw.empty:
    raise RuntimeError(
        "No series could be retrieved, and the cache in data/raw/ is empty.\n"
        "This note reads from a local cache so it never depends on a live\n"
        "network at render time. Populate it once with:\n"
        "    Rscript tools/seed_fred_cache.R\n"
        "then render again."
    )

status.drop(columns=["detail"])
```

```
Retrieved 12 of 12 series.
```

| series_id | result | n_obs | first | last |
|---|---|---|---|---|
| SP500 | ok | 2514 | 2016-08-15 | 2026-08-14 |
| T10Y3M | ok | 9161 | 1990-01-02 | 2026-08-14 |
| M2REAL | ok | 437 | 1990-01-01 | 2026-06-01 |
| PERMIT | ok | 438 | 1990-01-01 | 2026-06-01 |
| HOUST | ok | 438 | 1990-01-01 | 2026-06-01 |
| DGORDER | ok | 413 | 1992-02-01 | 2026-06-01 |
| UCOGNO | ok | 413 | 1992-02-01 | 2026-06-01 |
| AWHMAN | ok | 439 | 1990-01-01 | 2026-07-01 |
| ICSA | ok | 1910 | 1990-01-06 | 2026-08-08 |
| TEMPHELPS | ok | 439 | 1990-01-01 | 2026-07-01 |
| UMCSENT | ok | 438 | 1990-01-01 | 2026-06-01 |
| USSLIND | ok | 362 | 1990-01-01 | 2020-02-01 |

Two things in that table are worth stopping on.

First, check the `first` column against the `START` date we asked for. We requested 1990, but FRED does not serve 1990 for every series — `SP500`, for instance, is only distributed with a rolling ten-year window. **The start date you request is a ceiling, not a guarantee**, and a model that silently uses whatever it got back will be fit on a much shorter sample than you intended.

Second, any row marked `FAILED` is a series that FRED no longer publishes under that ID. Retired IDs are a normal fact of life with FRED, and the reason `fred_frame` fetches one at a time instead of making one bulk call. The `detail` column, which we hid above, carries the specific error for each.

```
print(f"{raw.shape[0]:,} rows x {raw.shape[1]} series")
print(f"Index spans {raw.index.min().date()} to {raw.index.max().date()}")
raw.tail(8)
```

```
11,528 rows x 12 series
Index spans 1990-01-01 to 2026-08-14
```

| date | SP500 | T10Y3M | M2REAL | PERMIT | HOUST | DGORDER | UCOGNO | AWHMAN | ICSA | TEMPHELPS | UMCSENT | USSLIND |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 2026-08-06 | 7709.96 | 0.79 | NaN | NaN | NaN | NaN | NaN | NaN | NaN | NaN | NaN | NaN |
| 2026-08-07 | 7757.64 | 0.78 | NaN | NaN | NaN | NaN | NaN | NaN | NaN | NaN | NaN | NaN |
| 2026-08-08 | NaN | NaN | NaN | NaN | NaN | NaN | NaN | NaN | 209000.0 | NaN | NaN | NaN |
| 2026-08-10 | 7753.11 | 0.83 | NaN | NaN | NaN | NaN | NaN | NaN | NaN | NaN | NaN | NaN |
| 2026-08-11 | 7728.20 | 0.81 | NaN | NaN | NaN | NaN | NaN | NaN | NaN | NaN | NaN | NaN |
| 2026-08-12 | 7748.50 | 0.81 | NaN | NaN | NaN | NaN | NaN | NaN | NaN | NaN | NaN | NaN |
| 2026-08-13 | 7798.99 | 0.76 | NaN | NaN | NaN | NaN | NaN | NaN | NaN | NaN | NaN | NaN |
| 2026-08-14 | 7785.76 | 0.82 | NaN | NaN | NaN | NaN | NaN | NaN | NaN | NaN | NaN | NaN |

That tail looks alarming — almost every cell is `NaN`. It is not an error. It is what an outer join across mixed publication frequencies looks like: the daily series create a row for every business day, and the monthly series have a value on only one of those rows per month.

## Reconciling the frequencies

This is the step the R note never had to confront, because it stopped before downloading. Any model using these indicators together needs them on one clock. The defensible choice is to step *down* to the slowest frequency — monthly — rather than forward-filling monthly data onto daily rows, which would fabricate precision the source data does not have.

How you aggregate depends on what the series measures:

- **Levels and indexes** (`SP500`, `T10Y3M`, `UMCSENT`, `AWHMAN`) — take the monthly *mean*, which is the standard convention for converting a high-frequency level to a monthly observation.
- **Flows** (`ICSA`, initial claims) — these are counts accumulated over a week, so a monthly *sum* is the meaningful aggregate.
- **Already monthly** — leave alone.

```
SUM_SERIES = {"ICSA"}

available = [c for c in raw.columns]
monthly = pd.DataFrame(index=raw.resample("MS").mean().index)

for sid in available:
    s = raw[sid].dropna()
    if fred_series[sid]["freq"] == "M":
        m = s.resample("MS").last()
    elif sid in SUM_SERIES:
        m = s.resample("MS").sum()
    else:
        m = s.resample("MS").mean()
```

```
    monthly[sid] = m

monthly = monthly.dropna(how="all")
print(f"{monthly.shape[0]:,} monthly rows x {monthly.shape[1]} series")
monthly.tail(6)
```

```
440 monthly rows x 12 series
```

| date | SP500 | T10Y3M | M2REAL | PERMIT | HOUST | DGORDER | UCOGNO | AWHMAN | ICSA | TEMPHELPS | UMCSENT | USSLIND |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 2026-03-01 | 6654.419091 | 0.526818 | 6865.4 | 1363.0 | 1522.0 | 320485.0 | 269892.0 | 41.5 | 832000.0 | 2479.9 | 53.3 | NaN |
| 2026-04-01 | 6957.007619 | 0.622727 | 6859.0 | 1423.0 | 1414.0 | 347767.0 | 266116.0 | 41.6 | 831000.0 | 2490.4 | 49.8 | NaN |
| 2026-05-01 | 7412.551500 | 0.798500 | 6903.3 | 1410.0 | 1199.0 | 333842.0 | 275725.0 | 41.6 | 1058000.0 | 2490.6 | 44.8 | NaN |
| 2026-06-01 | 7450.032857 | 0.659524 | 6962.5 | 1374.0 | 1427.0 | 335418.0 | 276997.0 | 41.7 | 890000.0 | 2501.6 | 49.5 | NaN |
| 2026-07-01 | 7481.339091 | 0.732727 | NaN | NaN | NaN | NaN | NaN | 41.7 | 813000.0 | 2505.0 | NaN | NaN |
| 2026-08-01 | 7734.273000 | 0.787000 | NaN | NaN | NaN | NaN | NaN | NaN | 409000.0 | NaN | NaN | NaN |

```
coverage = monthly.notna().sum().sort_values()

fig, ax = plt.subplots(figsize=(8, 4.5))
ax.barh(coverage.index, coverage.values, color=UNLV_SCARLET, alpha=0.85)
ax.set_xlabel("months of data")
ax.set_title("How much history do we actually have?")
for y, v in enumerate(coverage.values):
    ax.text(v, y, f" {v:,}", va="center", fontsize=8, color=UNLV_GRAY)
ax.grid(axis="y", visible=False)
fig.tight_layout()
plt.show()
```

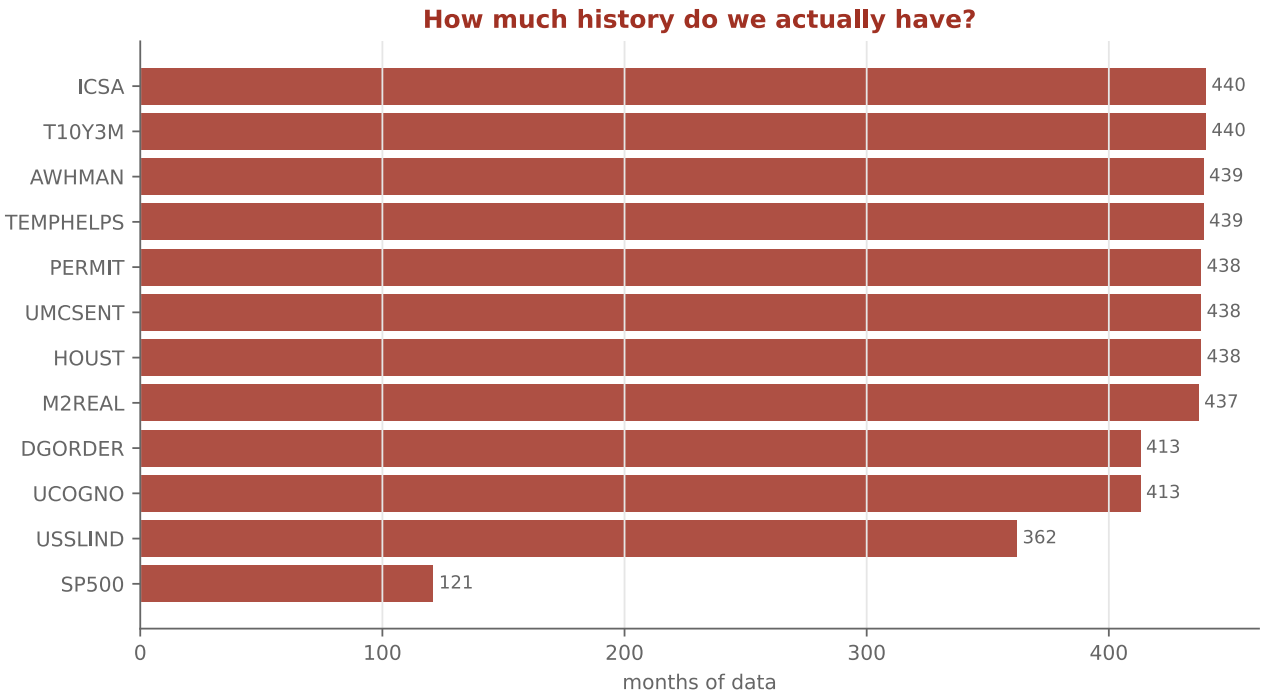



Figure 1: Non-missing observations by series, monthly frequency.

The complete-case sample — the rows where *every* series is present — is the binding constraint on any model that uses all of them at once:

```
complete = monthly.dropna()
if len(complete):
    print(f"Complete cases: {len(complete):,} months "
          f"({complete.index.min().date()} to {complete.index.max().date()})")
else:
    print("No month has all series present.")

print("\nLongest common window if we drop the shortest series:")
trimmed = monthly.drop(columns=[coverage.index[0]]).dropna()
if len(trimmed):
    print(f"  {len(trimmed):,} months "
          f"({trimmed.index.min().date()} to {trimmed.index.max().date()})")
```

```
Complete cases: 43 months (2016-08-01 to 2020-02-01)

Longest common window if we drop the shortest series:
  337 months (1992-02-01 to 2020-02-01)
```

## Looking at the indicators

```
cols = [c for c in monthly.columns if monthly[c].notna().any()]
ncol = 3
nrow = -(-len(cols) // ncol)

fig, axes = plt.subplots(nrow, ncol, figsize=(11, 2.3 * nrow), sharex=True)
axes = axes.ravel()

for ax, sid in zip(axes, cols):
    s = monthly[sid].dropna()
    ax.plot(s.index, s.values, color=UNLV_SCARLET)
    ax.set_title(f"{sid} — {fred_series[sid]['label']}", fontsize=8.5)
    ax.tick_params(labelsize=7)

for ax in axes[len(cols):]:
    ax.set_visible(False)

fig.tight_layout()
plt.show()
```

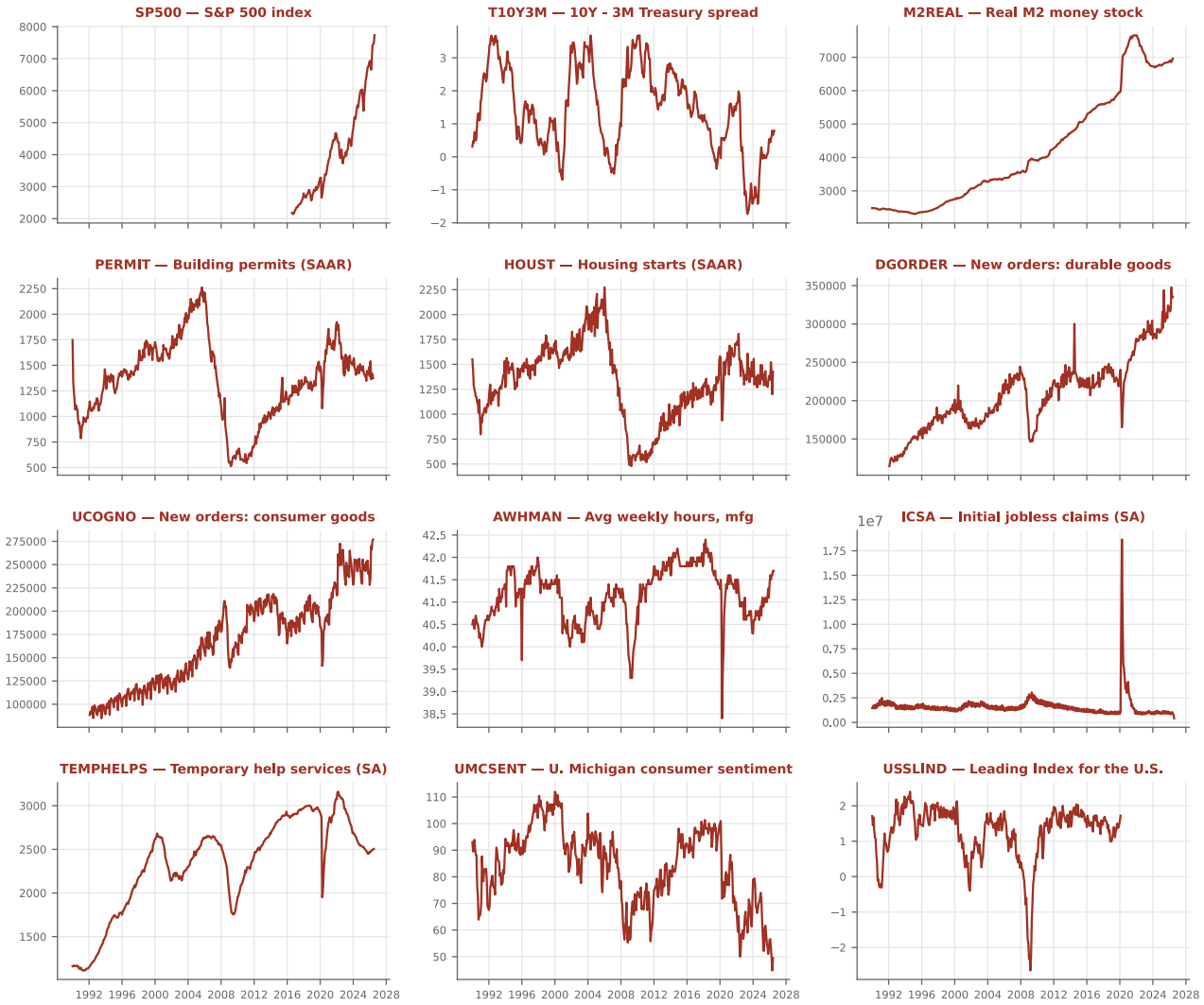



Figure 2: Leading indicators, monthly. Note the differing vertical scales and start dates.

Plotted on a common time axis, the differing start dates are obvious in a way the status table only hinted at. Several of these series simply do not exist for the early part of the window.

```
# How you measure "change" depends on what the series is.
#
# A percent change is only meaningful for a quantity that cannot be zero or
# negative. T10Y3M is a SPREAD: it crosses zero every time the yield curve
# inverts. A spread moving from -0.10 to +0.10 is not a "200% decline" -- that
# figure is arithmetic nonsense, and dividing by a value at or near zero
# produces infinities that poison everything downstream.
#
# For series that can cross zero, the meaningful change is a simple
# difference, in the series' own units.
crosses_zero = [c for c in monthly.columns if (monthly[c].dropna() <= 0).any()]

changes = pd.DataFrame(index=monthly.index)
for col in monthly.columns:
    if col in crosses_zero:
        changes[col] = monthly[col].diff()
    else:
```

```
        changes[col] = monthly[col].pct_change(fill_method=None)

changes = changes.replace([float("inf"), float("-inf")], float("nan"))

print(f"Percent changes : {[c for c in monthly.columns if c not in crosses_zero]}")
print(f"Differences     : {crosses_zero}  (these cross zero)")

corr = changes.corr(min_periods=24)

fig, ax = plt.subplots(figsize=(7.5, 6.5))
im = ax.imshow(corr.values, cmap="RdBu_r", vmin=-1, vmax=1)
ax.set_xticks(range(len(corr)), corr.columns, rotation=90, fontsize=8)
ax.set_yticks(range(len(corr)), corr.index, fontsize=8)
ax.grid(visible=False)
for i in range(len(corr)):
    for j in range(len(corr)):
        v = corr.values[i, j]
        if pd.notna(v):
            ax.text(j, i, f"{v:.2f}", ha="center", va="center", fontsize=6.5,
                    color="white" if abs(v) > 0.55 else "#333333")
fig.colorbar(im, ax=ax, shrink=0.8, label="correlation")
ax.set_title("Correlation of monthly percent changes")
fig.tight_layout()
plt.show()
```

```
Percent changes : ['SP500', 'M2REAL', 'PERMIT', 'HOUST', 'DGORDER', 'UCOGNO', 'AWHMAN', 'ICSA', 'TEMPHELPS',
'UMCSENT']
Differences     : ['T10Y3M', 'USSLIND']  (these cross zero)
```

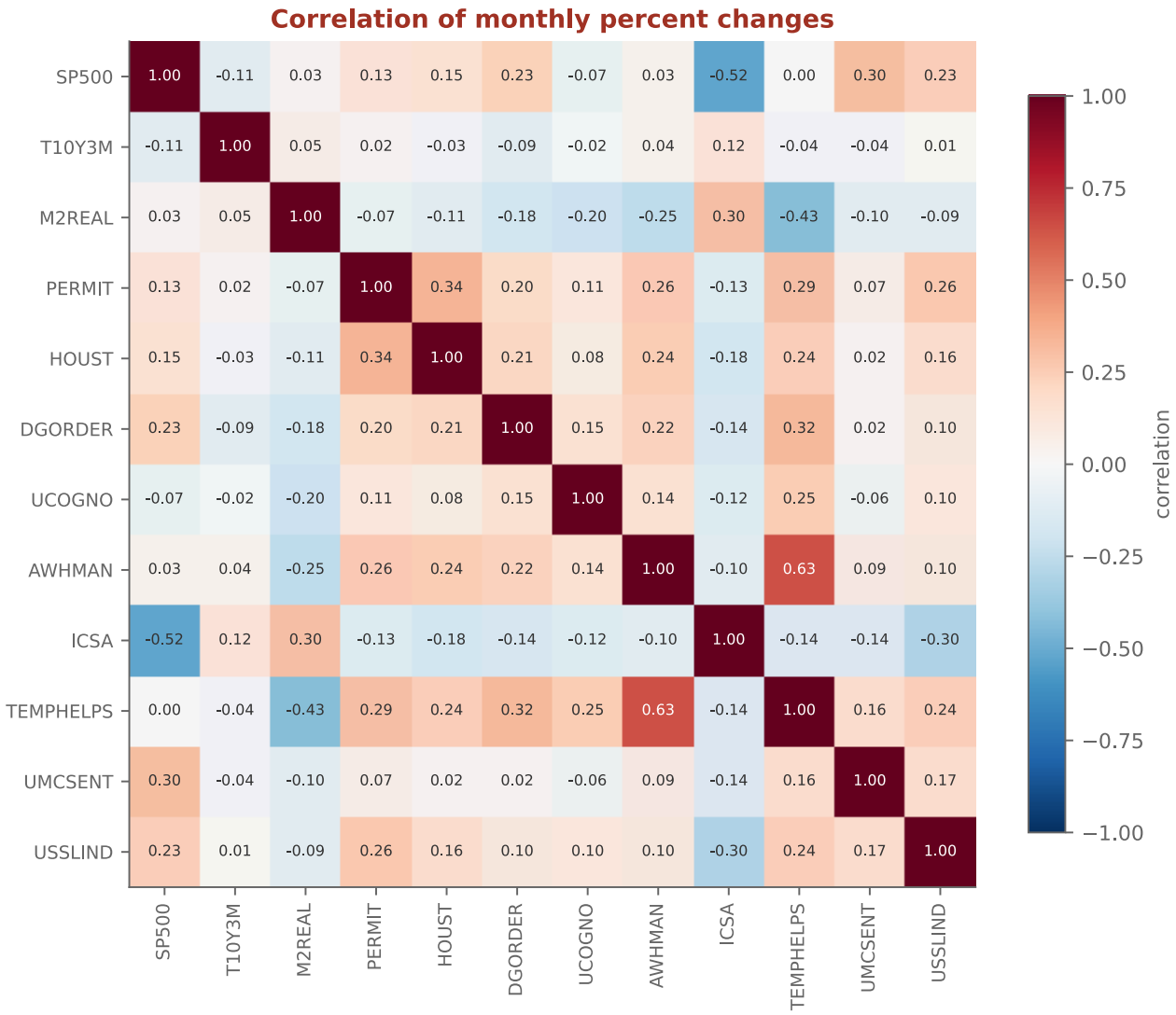



Figure 3: Pairwise correlation of month-over-month changes, complete cases only.

Correlating the *changes* rather than the levels matters here. Most of these series trend, and correlating trending levels will report near-perfect relationships between quantities that have nothing to do with one another.

But "change" is not one thing. For an index or a level that stays positive, a percent change is the natural measure. For a **spread**, it is meaningless: `T10Y3M` is the gap between two yields and it goes negative whenever the curve inverts. Ask for a percent change there and you get figures like "−200%" for a move from −0.10 to +0.10, plus an infinity every time the denominator touches zero. One bad column is enough to make the entire correlation matrix uncomputable.

> Before transforming a variable, ask what it measures. A ratio, a level, a spread, and a count do not all admit the same operations, and the code will not stop you from applying the wrong one.

## What to take from this

1. The series IDs are the easy part. Reconciling **frequency** and **coverage** is where the actual work is, and it is invisible until you download.
2. `start=` is a request, not a promise. Always verify the window you got.

3. Aggregate flows by summing and levels by averaging — the choice is determined by what the series measures, not by convenience.
4. Correlate changes, not levels, unless you have a reason to do otherwise.